Wind Turbine Project

Rehand A.Yassin 202000770

University of Science and Technology at Zewail City

REE 430: Rotor Aerodynamics

Professor Mohamed Madbouli

May, 2025

Table of contents

Abstract

With the global shift toward sustainable energy, wind power has emerged as a key player in the renewable energy sector. At the heart of any wind turbine is the blade — a component whose aerodynamic design has a direct impact on the turbine's efficiency and energy output. This project explores the aerodynamic optimization of wind turbine blades using the NACA 23015 airfoil, known for its strong lift-to-drag characteristics.

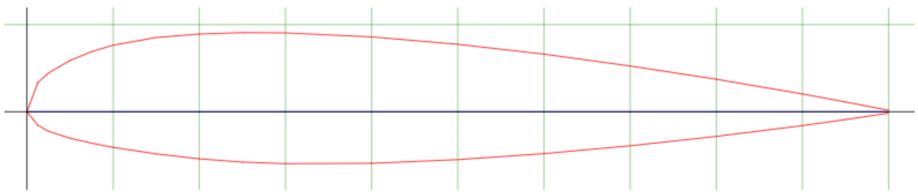
The study is carried out using MATLAB and QBlade. MATLAB is used for detailed aerodynamic calculations, including lift and drag analysis across a range of angles of attack, determining the best angle for maximum lift-to-drag ratio, optimizing chord and twist distributions, and evaluating the blade's performance under off-design conditions. QBlade complements this by providing simulation-based insights through blade element momentum (BEM) analysis, structural checks, and dynamic performance evaluations under variable wind speeds.

1. Introduction

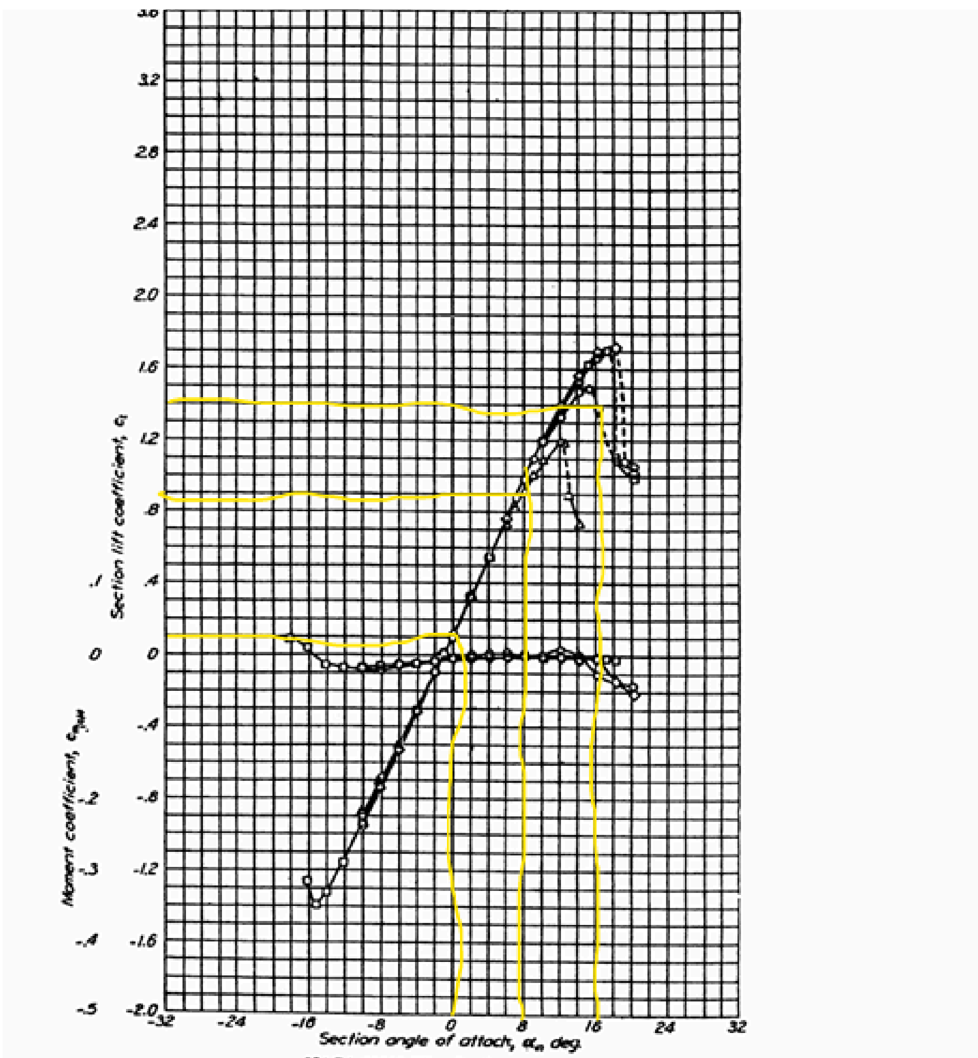
The primary objective of this study is to develop a comprehensive understanding of the aerodynamic optimization process for horizontal-axis wind turbine blades, with a focus on the NACA 23015 airfoil. The project integrates computational modeling in MATLAB with simulation results from QBlade to achieve a robust and well-validated blade design. This dual-tool methodology allows for:

- Accurate verification of key aerodynamic and structural parameters to ensure optimal performance,
- Cross-validation between analytical and simulation-based approaches enhances the reliability and consistency of the design process.
- Improved adaptability and durability of the wind turbine blade across a wide range of operating conditions, including variations in wind speed and loading scenarios.
- By combining theoretical modeling with practical simulation, the study aims to produce a high-efficiency blade design that meets both performance and structural requirements for modern wind energy systems.

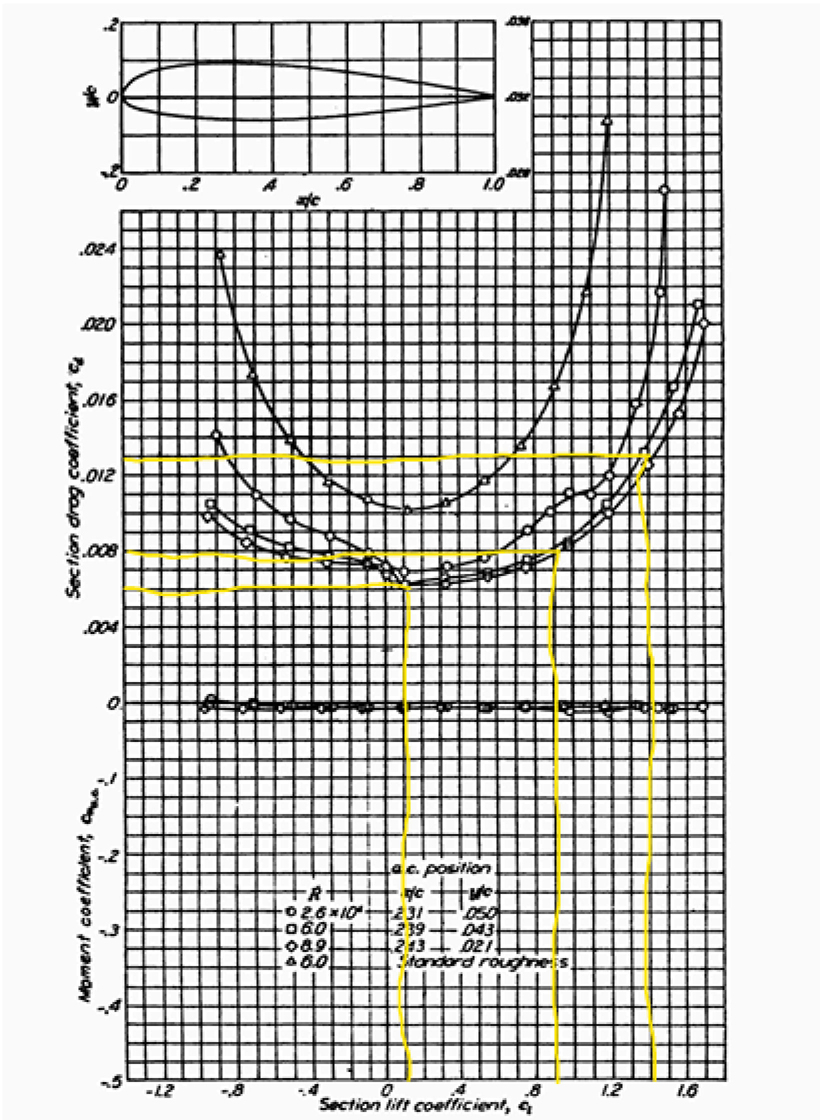
2. NACA 23015 airfoil (naca23015-il)



3. Calculations to find C_l & C_d Constants



NACA 23015 Wing Section



NACA 23015 Wing Section (Continued)

From this equation

$$C_l = C_{lo} + C_{l\alpha}\alpha \qquad \& \; C_d = C_{do} + k_1 C_l + k_2 C_l^2$$

And from graphs

$$\alpha = 0 \;\Rightarrow\; C_l = 0.1 \;\&\; C_d = 0.006$$

$$\alpha = 8 \;\Rightarrow\; C_l = 0.9 \;\&\; C_d = 0.008$$

$$\alpha = 16 \;\Rightarrow\; C_l = 1.4 \;\&\; C_d = 0.013$$

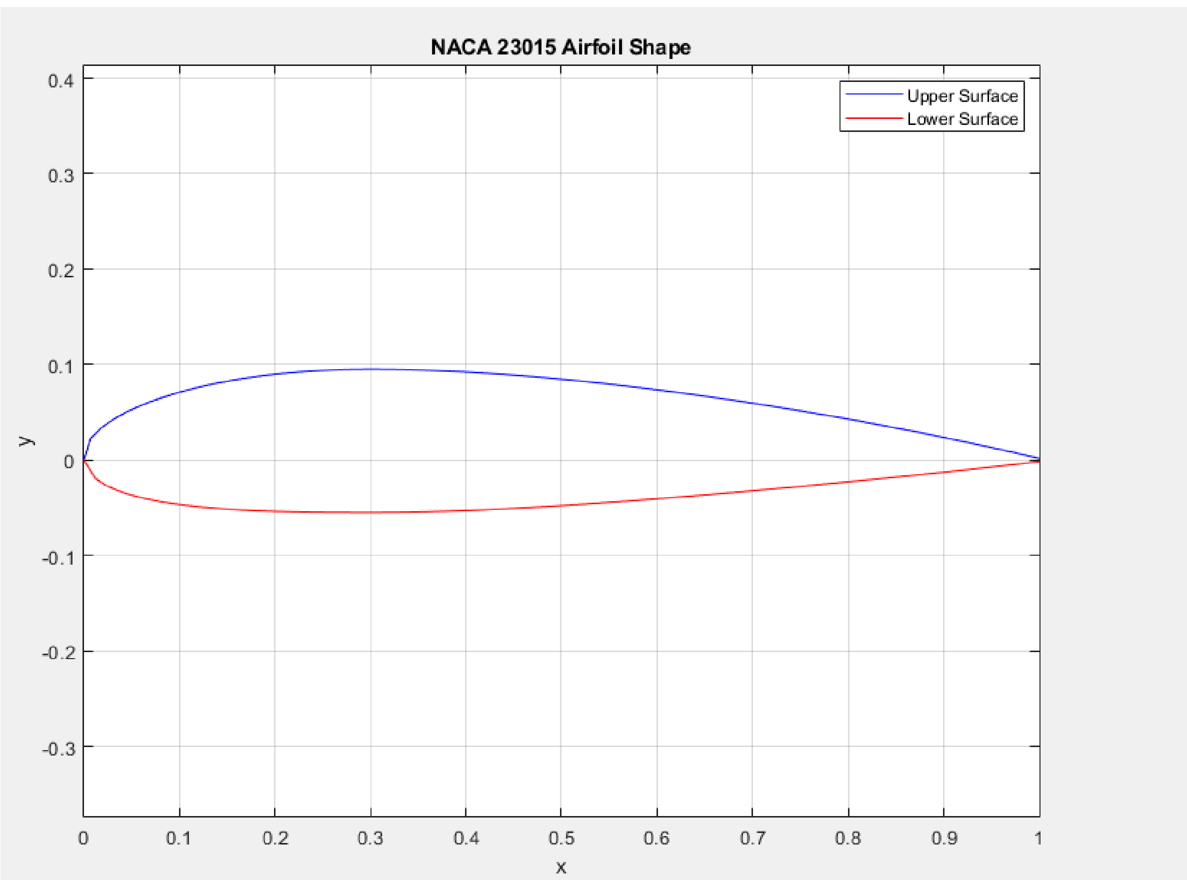
$C_{do} = 0.010393$ $k_1 =- 0.0003751$ $k_2 = 0.0107141$ $C_{lo} = 0.121645$
$C_{l\alpha} = 0.109928$
Substitute in

$$C_{l_{\left(\frac{C_l}{C_d}\right)_{max}}} = \sqrt{\frac{C_{do}}{k_2}} = 0.9489$$

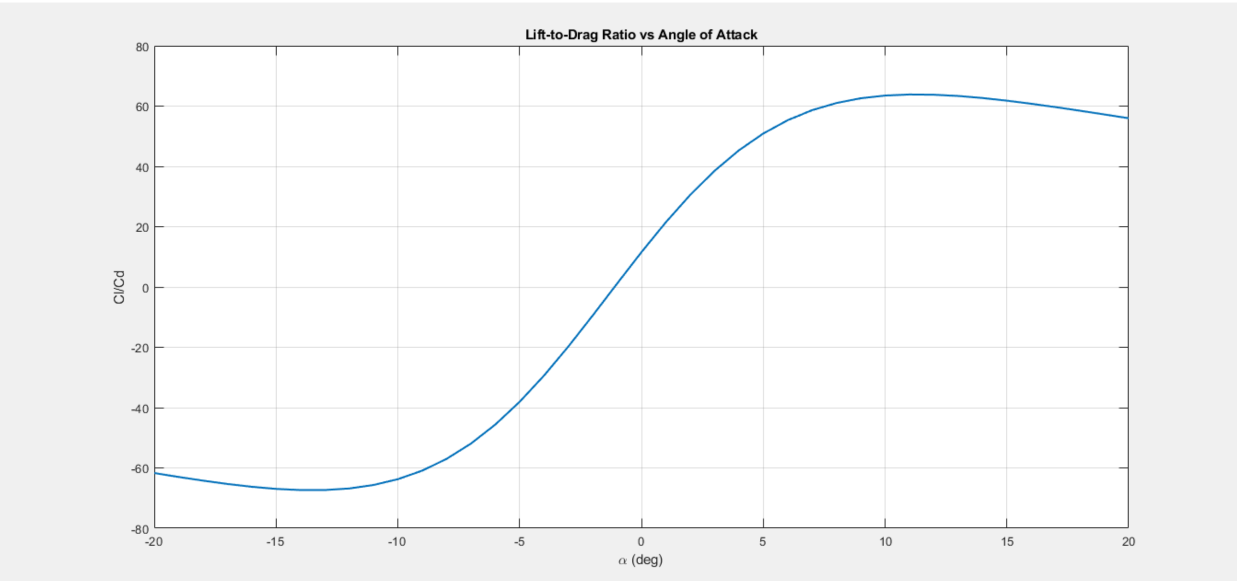
So $\alpha_{max, design} = 9^{o}$

4. MATLAB Results

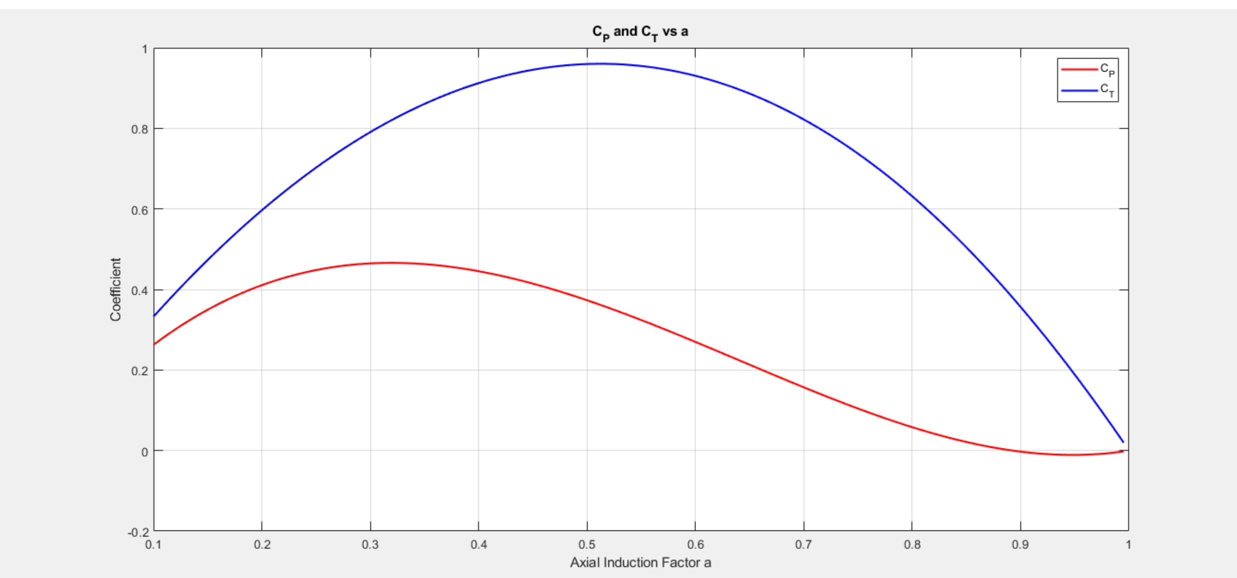
4.1. 2D Airfoil



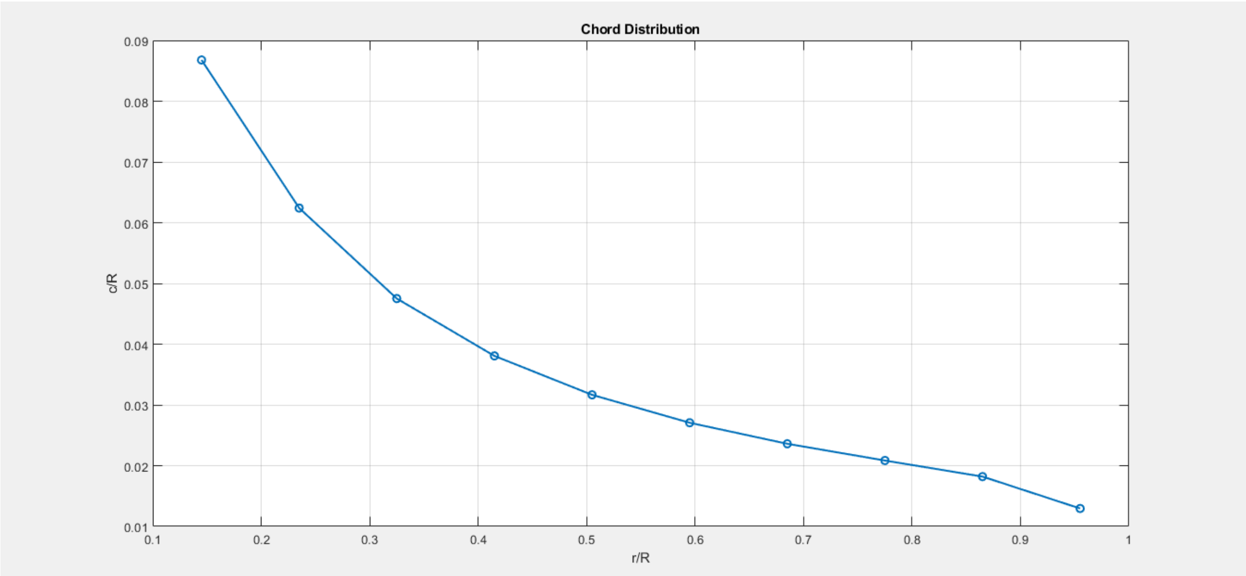
4.2. Design Problem



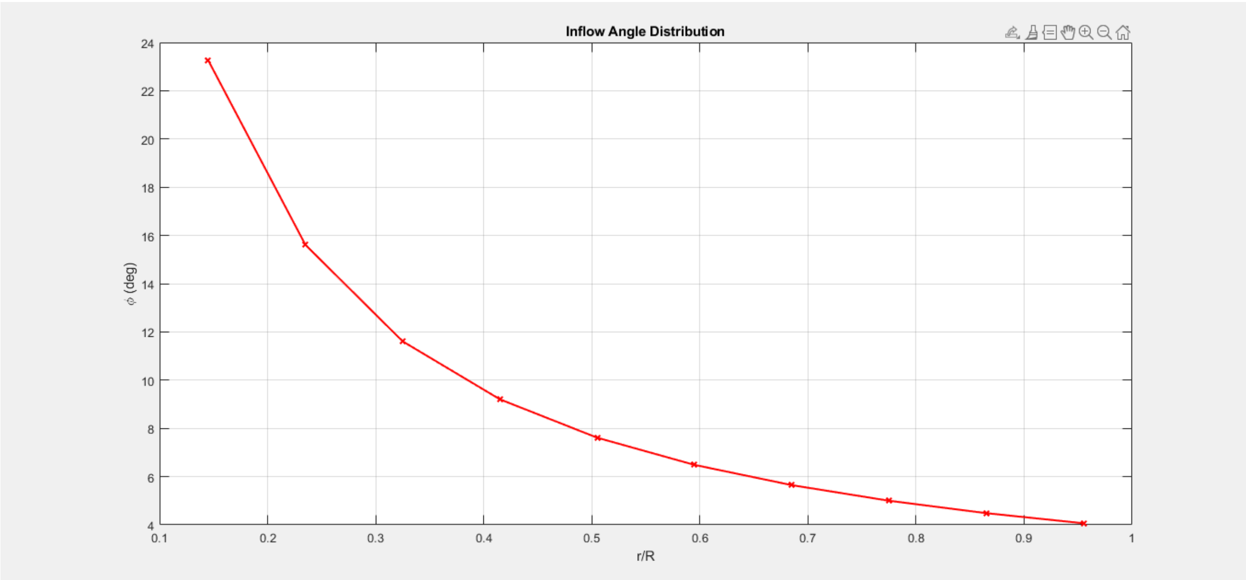
- Peak at α ≈ 9° confirms optimal performance.
- Sharp drop beyond 10° → stall onset.
- Negative α → highly inefficient (Cl/Cd < 0).



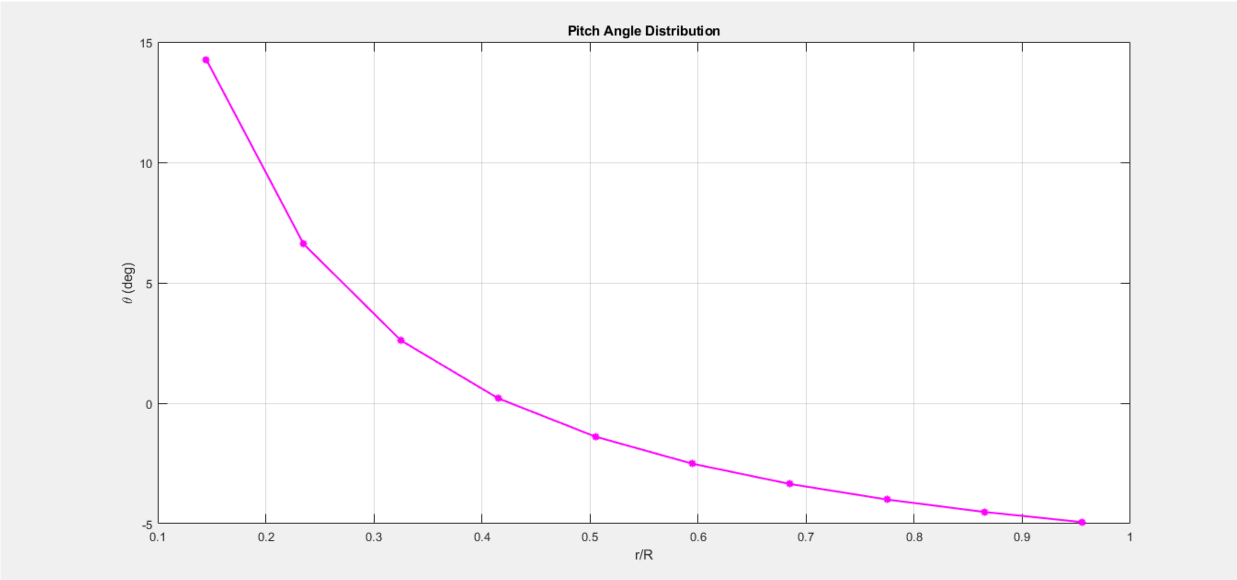
- Cp peaks at a ≈ 0.3–0.4 → ideal axial loading.
- Ct increases with a but drops after peak → high a leads to reduced efficiency.



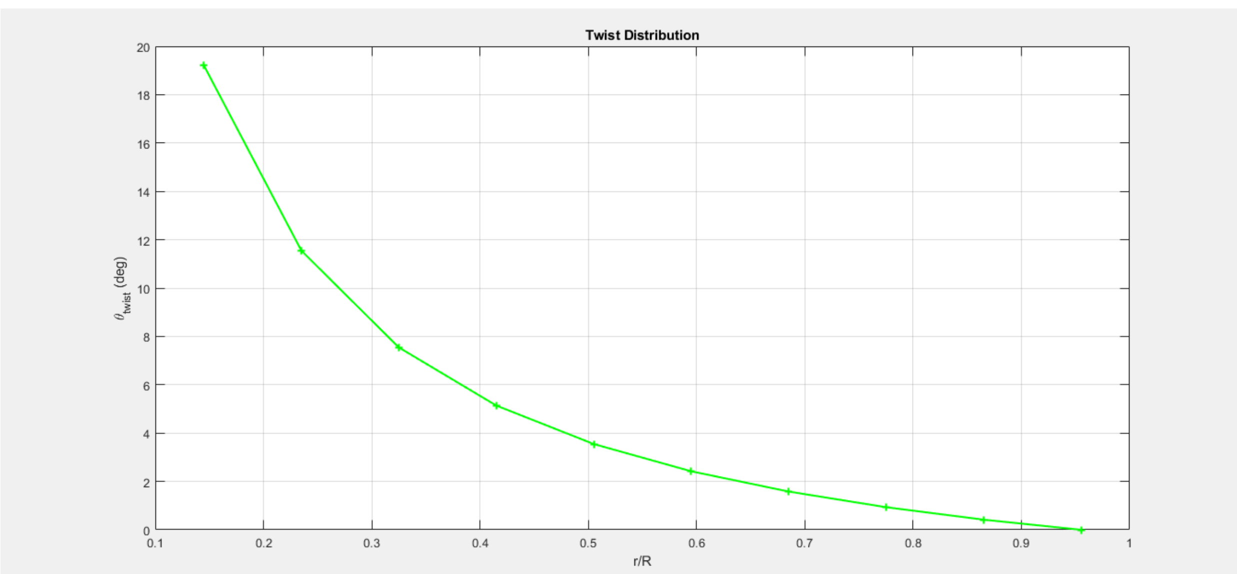
- The chord that is largest near the hub → handles the low-speed region.
- Smooth tapering toward the tip → reduces drag and improves efficiency.



- High near hub due to low λr.
- Decreases toward the tip → flatter flow near the outer span.

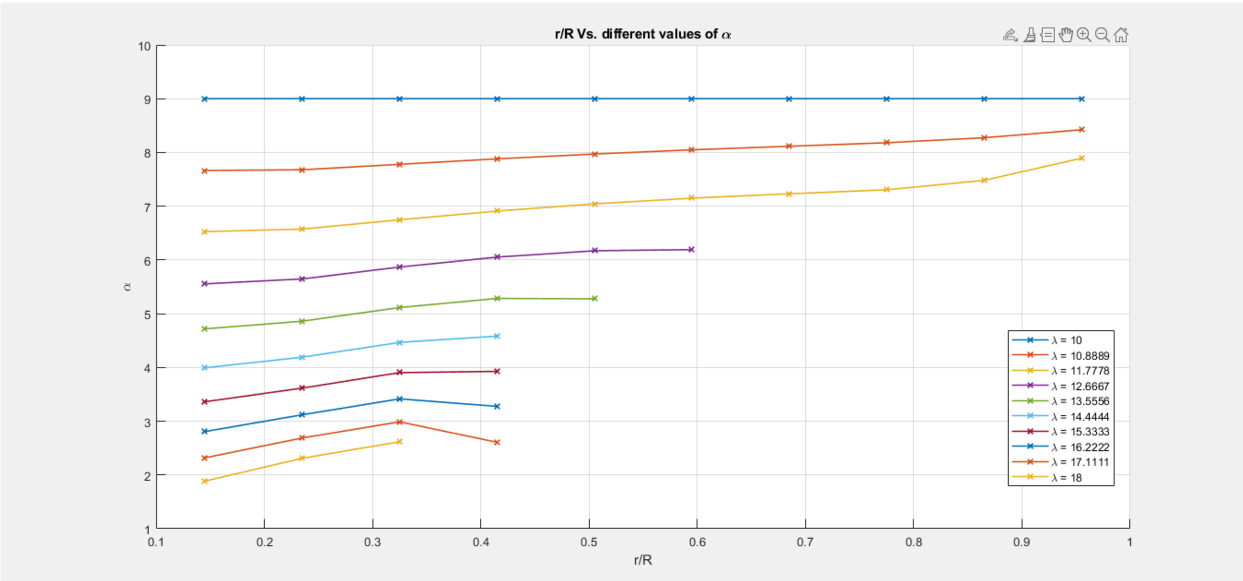


- Highest at the root to maintain the angle of attack.
- Falls to near 0° at tip → aligns with freestream.

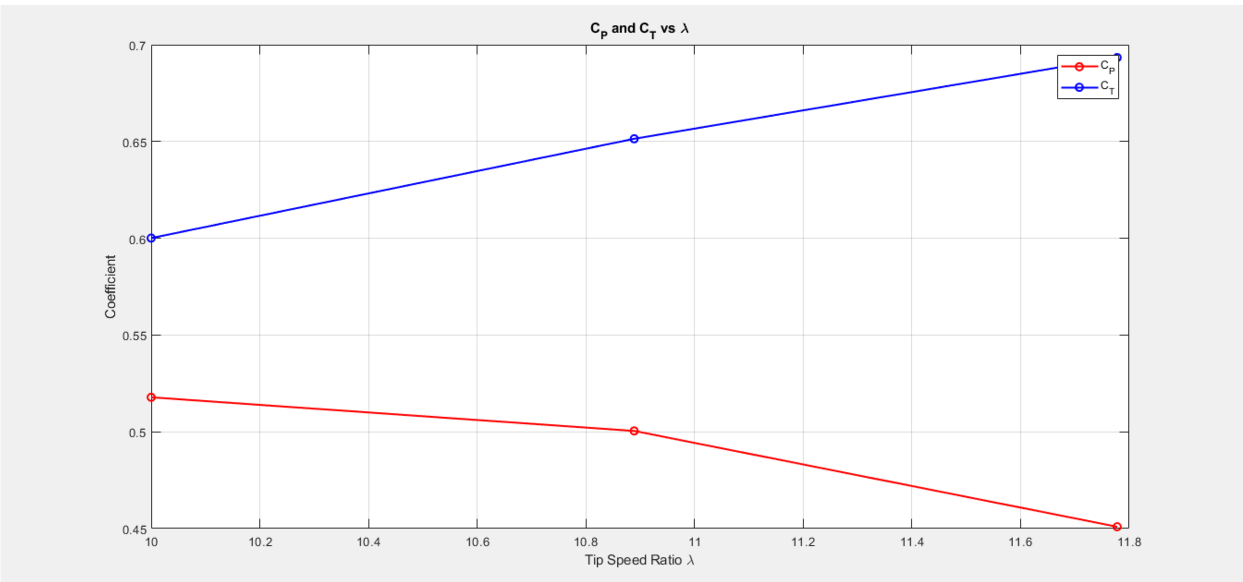


- High twist at root (≈19°), drops to ≈0° → aerodynamic matching across the blade.
- Indicates aggressive shape near hub.

4.3. Off-Design Problem



- Higher λ → lower α.
- α increases at the inner span across all λ due to reduced λr.
- Risk of stall for low λ if α > 10°.



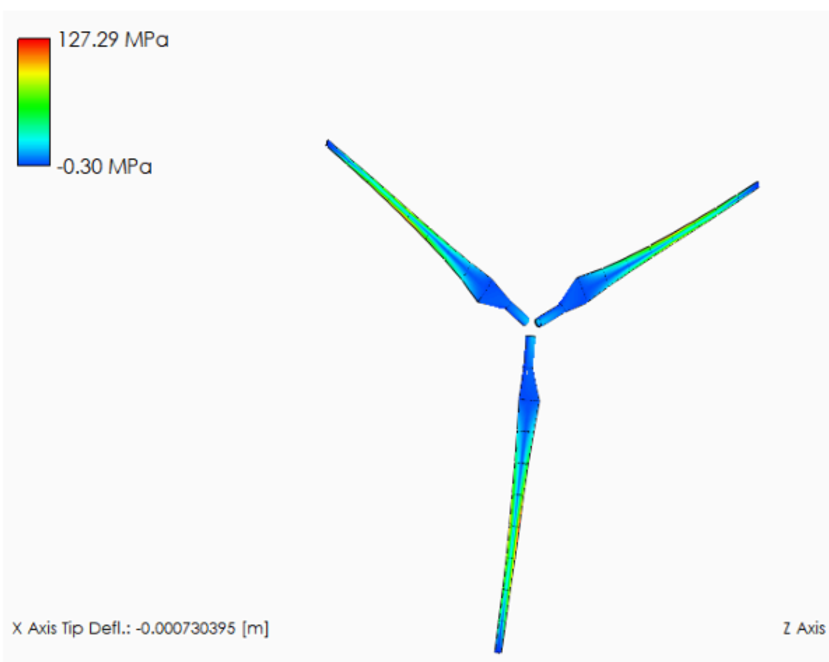
- Cp peaks and drops after λ ≈ 10.5 → tip speed too high = overloading.
- Ct increases with λ → more thrust from faster rotor speed.

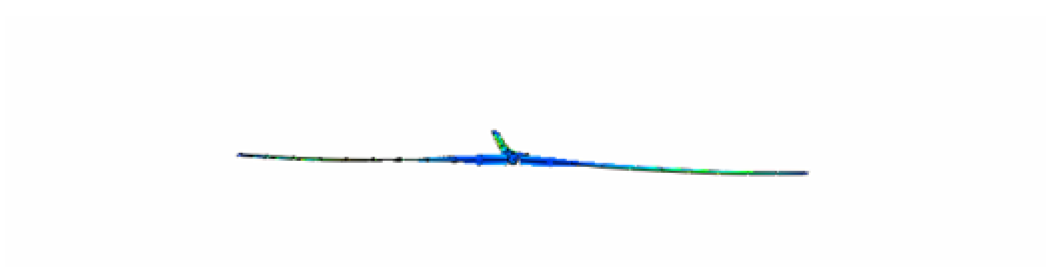
4.4. Off-Design Aerodynamic Performance Table

Lambda	Old_Cl	Old_Cd	Phi_deg	New_Cl	Corrected_New
10	1.111	0.017752	9.2983	1.111	0.017752
10.889	1.0013	0.016415	8.3008	1.0011	0.016415
11.778	0.90054	0.015309	7.3838	0.90071	0.015309
12.667	0.77177	0.014045	9.2117	0.77178	0.014045
13.556	0.67685	0.013236	9.5106	0.6769	0.013236
14.444	0.59523	0.012622	10.23	0.59523	0.012622
15.333	0.52856	0.012176	9.6233	0.52862	0.012176
16.222	0.46816	0.011814	9.0739	0.46863	0.011814
17.111	0.41283	0.011519	8.5705	0.41316	0.011519
18	0.37126	0.011322	10.098	0.37126	0.011322

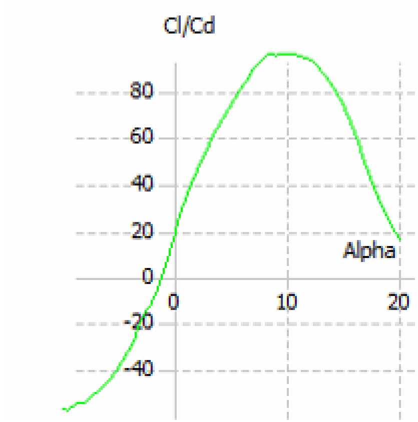
5. Q-Blade Results

5.1. 3D Blade Design

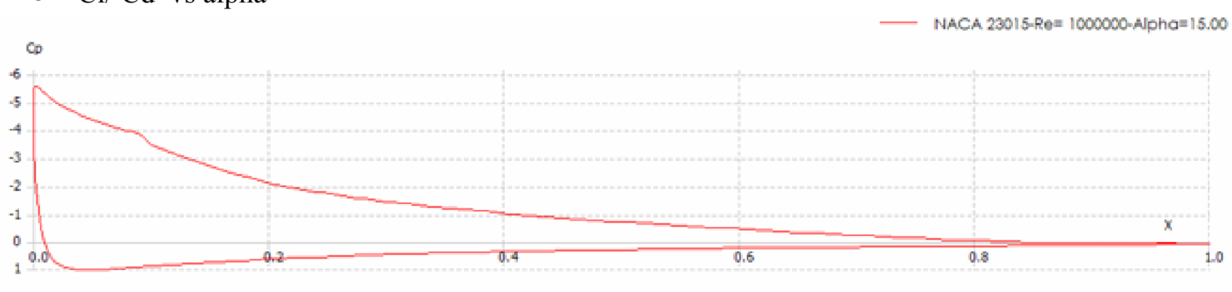




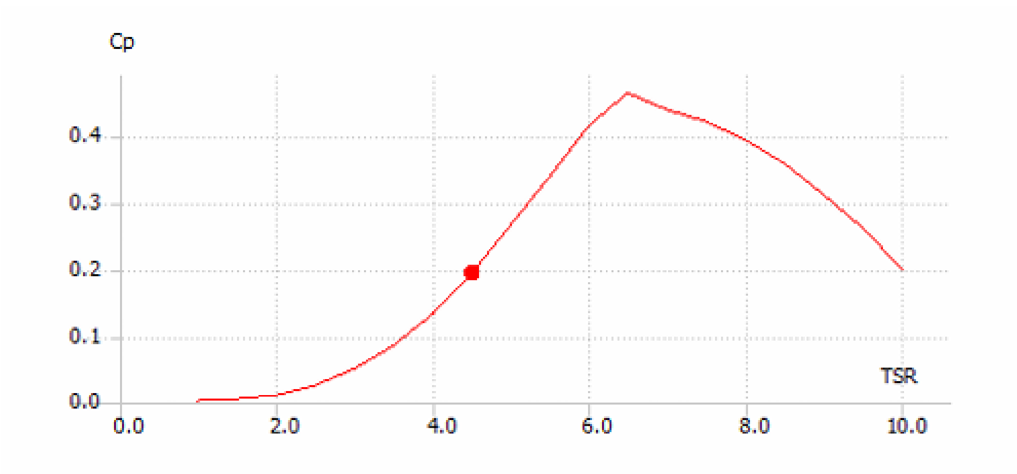
5.2. Plots



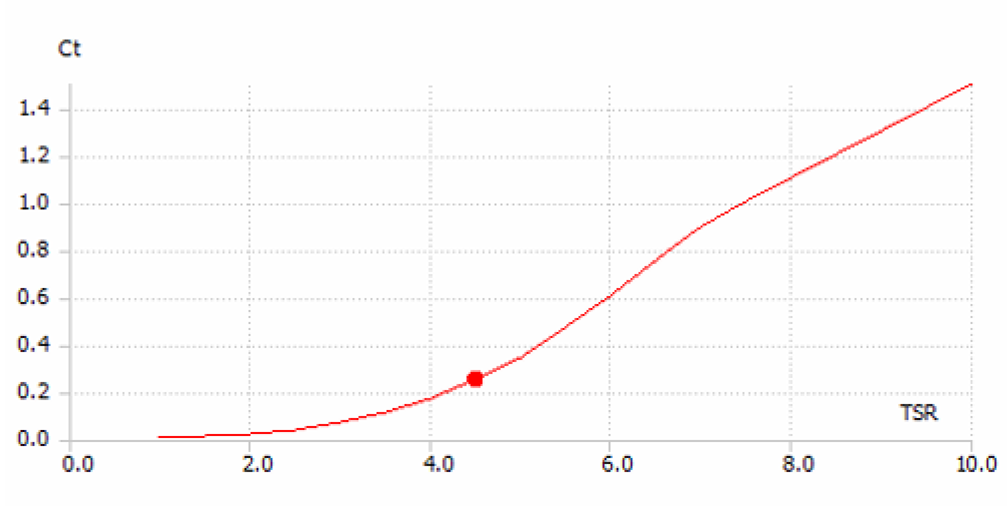
- Cl/ Cd vs alpha



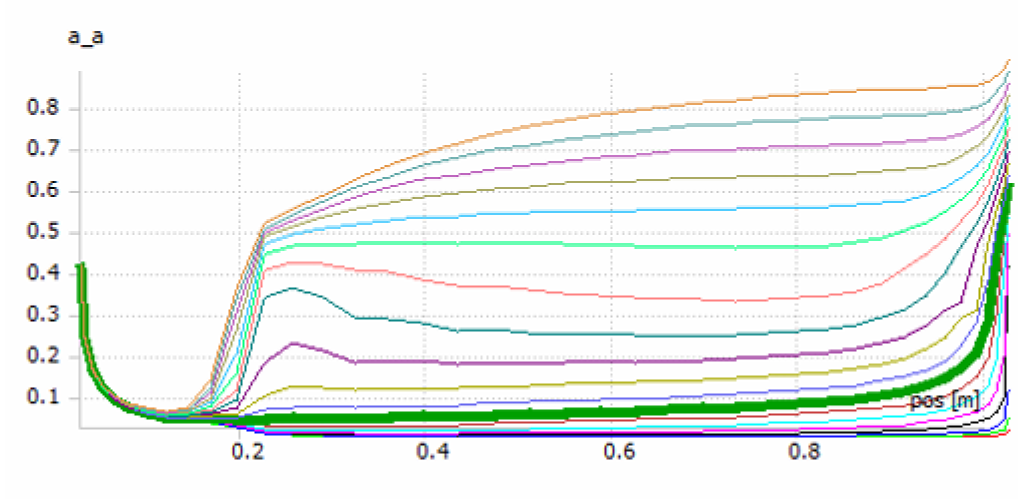
- Cp vs alpha



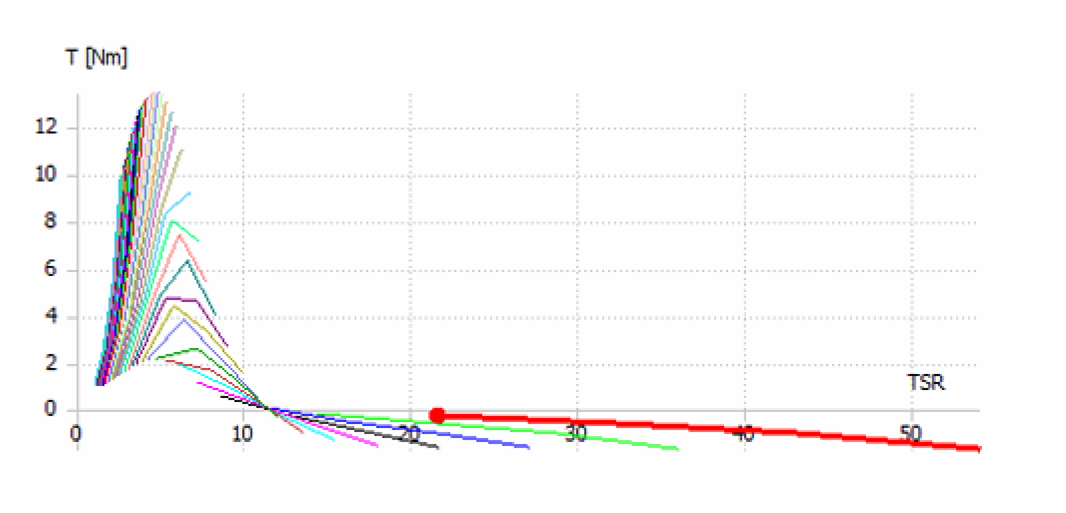
- Cp vs tip speed ratio



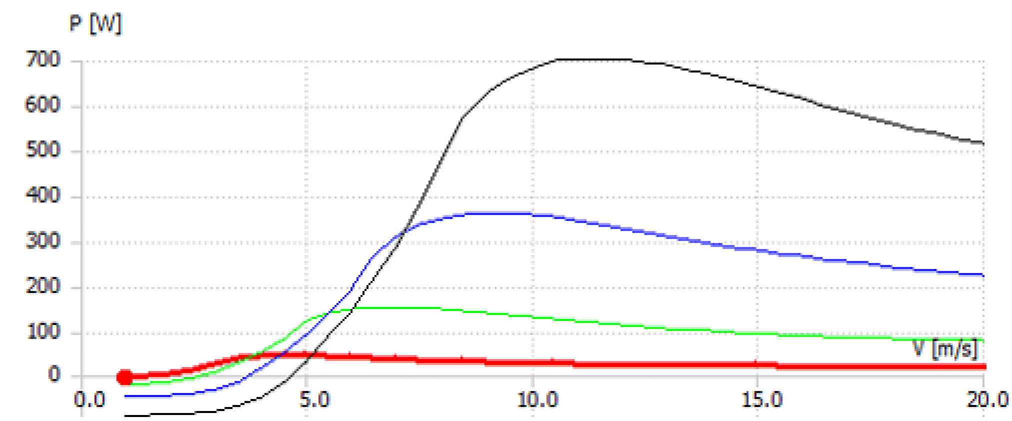
- Ct vs tip to speed ratio



- Axial induction factor vs Blade span position



- Thrust vs Tip speed ratio



- Power vs velocity

6. Conclusion

This project successfully designed and analyzed a horizontal-axis wind turbine blade using BEM theory and aerodynamic optimization. A smooth NACA 23015 airfoil was interpolated with cosine spacing and integrated into QBlade. The blade geometry was evaluated for chord, twist, and aerodynamic loading. Simulation results showed stable axial induction values and efficient performance across a range of tip-speed ratios. The final blade meets design targets with optimized aerodynamic behavior along the span.

7. References

[1] I. H. Abbott and A. E. von Doenhoff, *Theory of Wing Sections: Including a Summary of Airfoil Data*, Dover Publications, 1959.

[2] "Airfoil Tools – Airfoil Plotter," [Online]. Available: http://airfoiltools.com/plotter/index.

8. Appendix

8.1. MATLAB Code

```
clc; clear; close all;
%% Design Parameters
B = 3; N = 10;
lambda_design = 10;
alpha_max = 9;
C_do =  0.010393;
cl1 = 0.397278911564626;
cl2 = 0.6489795918367351;
cd1 = 0.011438596491228071;
cd2 = 0.013014643145393747;
syms K1 K2
E1 = [cd1 == C_do + K1*cl1 + K2*cl1^2, cd2 == C_do + K1*cl2 + K2*cl2^2];
[k1_temp ,k2_temp] = solve(E1, K1, K2);
K_1 = double(k1_temp);
K_2 = double(k2_temp);
C_lo = 0.121645;
C_l_alpha = 0.109928;
C_l = C_lo + C_l_alpha * alpha_max;
C_d = C_do + K_1 * C_l + K_2 * C_l^2;
K = C_l / C_d;
a_range = 0.1:0.005:1;
%% Plot NACA 23015 Airfoil
x = linspace(0, 1, 100);
m = 2/100; p = 3/10; t = 15/100;
yt = 5*t*(0.2969*sqrt(x) - 0.1260*x - 0.3516*x.^2 + 0.2843*x.^3 - 0.1015*x.^4);
yc = zeros(size(x)); dyc_dx = zeros(size(x));
for i = 1:length(x)
  if x(i) < p
     yc(i) = m/p^2*(2*p*x(i) - x(i)^2);
     dyc_dx(i) = 2*m/p^2*(p - x(i));
```

```
  else
      yc(i) = m/(1-p)^2*((1 - 2*p) + 2*p*x(i) - x(i)^2);
      dyc_dx(i) = 2*m/(1 - p)^2*(p - x(i));
  end
end
theta = atan(dyc_dx);
xu = x - yt.*sin(theta); yu = yc + yt.*cos(theta);
xl = x + yt.*sin(theta); yl = yc - yt.*cos(theta);
figure;
plot(xu, yu, 'b', xl, yl, 'r'); axis equal;
title('NACA 23015 Airfoil Shape'); grid on;
xlabel('x'); ylabel('y'); legend('Upper Surface','Lower Surface');
% Optimal alpha search
alpha_range = -20:20;
Cl_range = 0.12164529246168299 + 0.1099277187 * alpha_range;
Cd_range = C_do + K_1 .* Cl_range + K_2 .* Cl_range.^2;
ClCd_range = Cl_range ./ Cd_range;
[~, idx_opt] = max(ClCd_range);
alpha_max_opt = alpha_range(idx_opt);
Cl_opt = Cl_range(idx_opt);
Cd_opt = Cd_range(idx_opt);
% Plot Cl/Cd vs alpha
figure;
plot(alpha_range, ClCd_range, 'LineWidth', 1.5);
xlabel('\alpha (deg)'); ylabel('Cl/Cd'); grid on;
title('Lift-to-Drag Ratio vs Angle of Attack');
% Preallocate
r_over_R = linspace(0.1 + 0.9/(2*N), 1-0.9/(2*N), N)';
lambda_r = r_over_R * lambda_design;
theta_tip = zeros(length(a_range), 1);
Results = [];
% Design Loop
for j = 1:length(a_range)
  a = a_range(j);
  [a_dash, phi, F, C_over_R, theta] = deal(zeros(N,1));

  for i = 1:N
      t1 = lambda_r(i) * (lambda_r(i)*K + 1);
      t2 = lambda_r(i)^2 * K;
      t0 = lambda_r(i)*a + K*a*(a-1);

      a_dash(i) = (-t1 + sqrt(t1^2 - 4*t0*t2)) / (2*t2);
      phi(i) = atand((1-a)/((1+a_dash(i))*lambda_r(i)));
```

```
        exp_arg = -(B/2)*(1-r_over_R(i))./(r_over_R(i)*sind(phi(i)));
        F(i) = (2/pi) * acos(exp(exp_arg));

        numerator = F(i) * 4 * a_dash(i) * 2*pi*r_over_R(i)*lambda_r(i) * sind(phi(i))^2;
        denominator = B*(1-a)*(C_l*sind(phi(i)) - C_d*cosd(phi(i)));
        C_over_R(i) = numerator / denominator;

        theta(i) = phi(i) - alpha_max;
    end

    theta_tip(j) = theta(end);
    theta_tewist(:,j) = theta - theta_tip(j);
    C_p(:,j) = (B/pi) * lambda_design * (1-a).^2 ./ (sind(phi).^2) .* ...
            (C_l*sind(phi) - C_d*cosd(phi)) .* C_over_R .* r_over_R * (0.9/N);
    C_T(:,j) = (B/pi) * (1-a).^2 ./ (sind(phi).^2) .* ...
            (C_l*cosd(phi) + C_d*sind(phi)) .* C_over_R * (0.9/N);
end
% Final Coefficients
C_P_final = sum(C_p, 1)';
C_T_final = sum(C_T, 1)';
% Find a_design
[~, idx_max] = max(C_P_final);
a_design = a_range(idx_max);
%% Detailed Blade Geometry at a_design
phi_design = zeros(N,1);
F_design = zeros(N,1);
C_over_R_design = zeros(N,1);
theta_design = zeros(N,1);
a_dash_design = zeros(N,1);
for i = 1:N
    t1 = lambda_r(i)*(lambda_r(i)*K + 1);
    t2 = lambda_r(i)^2*K;
    t0 = lambda_r(i)*a_design + K*a_design*(a_design-1);

    a_dash_design(i) = (-t1 + sqrt(t1^2 - 4*t0*t2)) / (2*t2);
    phi_design(i) = atand((1-a_design)/((1+a_dash_design(i))*lambda_r(i)));

    exp_arg = -(B/2)*(1-r_over_R(i))/(r_over_R(i)*sind(phi_design(i)));
    F_design(i) = (2/pi)*acos(exp(exp_arg));

    numerator = F_design(i)*4*a_dash_design(i)*2*pi*r_over_R(i)*lambda_r(i)*sind(phi_design(i))^2;
    denominator = B*(1-a_design)*(C_l*sind(phi_design(i)) - C_d*cosd(phi_design(i)));
    C_over_R_design(i) = numerator / denominator;
```

```
  theta_design(i) = phi_design(i) - alpha_max;
end
theta_tip_design = theta_design(end);
theta_twist_design = theta_design - theta_tip_design;
%% Summary Table
alpha_design = alpha_max*ones(N,1);
Results = table(r_over_R, C_over_R_design, phi_design, alpha_design, theta_design, theta_twist_design);
%% Off-Design Performance
Lamda_d = linspace(10,18,10);
Cp = zeros(length(Lamda_d),1);
Ct = zeros(length(Lamda_d),1);
omega_r = 0.5;
for j = 1:length(Lamda_d)
  lamda_r_off = r_over_R * Lamda_d(j);
  alpha_off = alpha_design;
  err = 1;

  while max(err) > 0.01
    Cl = C_lo + C_l_alpha * alpha_off;
    Cd = C_do + K_1*Cl + K_2*Cl.^2;
    k = Cl./Cd;

    phi_off = theta_design + alpha_off;
    phi_off_rad = deg2rad(phi_off);

    a_dash_off = (((k .* tan(phi_off_rad)) - 1) .* (1 - lamda_r_off .* tan(phi_off_rad))) ./ ...
            ((lamda_r_off .* k) .* (1 + tan(phi_off_rad).^2));
    a_off = 1 - lamda_r_off .* tan(phi_off_rad) .* (1 + a_dash_off);

    exp_arg = -(B/2)*(1-r_over_R)./(r_over_R.*sin(phi_off_rad));
    F = (2/pi)*acos(exp(exp_arg));

    tt2 = K_2 .* sin(phi_off_rad);
    tt1 = cos(phi_off_rad) + K_1.*sin(phi_off_rad);
    tt0 = C_do.*sin(phi_off_rad) - (8*pi*F.*a_off.*r_over_R.*(sin(phi_off_rad)).^2) ./ ...
       (B*C_over_R_design.*(1-a_off));

    disc = max(tt1.^2 - 4*tt0.*tt2,0);
    Cl_new = (-tt1 + sqrt(disc)) ./ (2*tt2);

    alpha_c = (Cl_new - C_lo)/C_l_alpha;
    alpha_n = omega_r*alpha_c + (1-omega_r)*alpha_off;

    err = abs(alpha_n - alpha_off);
```

```
        alpha_off = alpha_n;
    end

    Cp(j) = sum((B/pi)*Lamda_d(j)*((1-a_off).^2)./(sin(phi_off_rad).^2) .* ...
            (Cl.*sin(phi_off_rad) - Cd.*cos(phi_off_rad)) .* ...
            C_over_R_design .* r_over_R) / N;

    Ct(j) = sum((B/pi)*((1-a_off).^2)./(sin(phi_off_rad).^2) .* ...
            (Cl.*cos(phi_off_rad) + Cd.*sin(phi_off_rad)) .* ...
            C_over_R_design .* r_over_R) / N;
end
%% Create Off-Design Results Table
results = nan(length(Lamda_d), 6);
for j = 1:length(Lamda_d)
    lam = Lamda_d(j);
    lamda_r_off = r_over_R * lam;
    alpha_Off = alpha_design;
    error = ones(N, 1);

    while max(error) > 0.01
        Cl = C_lo + C_l_alpha * alpha_Off;
        Cd = C_do + K_1 * Cl + K_2 * Cl.^2;
        k = Cl ./ Cd;
        phi_off = theta_design + alpha_Off;
        phi_rad = deg2rad(phi_off);

        a_dash = (((k .* tan(phi_rad)) - 1) .* (1 - lamda_r_off .* tan(phi_rad))) ./ ...
            ((lamda_r_off .* k) .* (1 + tan(phi_rad).^2));
        a = 1 - lamda_r_off .* tan(phi_rad) .* (1 + a_dash);

        exp_arg = -(B/2)*(1 - r_over_R)./(r_over_R.*sin(phi_rad));
        F = (2/pi) * acos(exp(exp_arg));

        tt2 = K_2 .* sin(phi_rad);
        tt1 = cos(phi_rad) + K_1 .* sin(phi_rad);
        tt0 = C_do .* sin(phi_rad) - ...
            (8*pi*F.*a.*r_over_R.*(sin(phi_rad)).^2) ./ ...
            (B*C_over_R_design.*(1-a));

        disc = tt1.^2 - 4 * tt0 .* tt2;
        valid_idx = disc >= 0 & abs(tt2) > eps;

        Cl_new = NaN(size(tt1));
        Cl_new(valid_idx) = (-tt1(valid_idx) + sqrt(disc(valid_idx))) ./ (2*tt2(valid_idx));
```

```
        alpha_c = (Cl_new - C_lo) / C_l_alpha;
        alpha_n = omega_r * alpha_c + (1 - omega_r) * alpha_Off;
        error = abs(alpha_n - alpha_Off);
        alpha_Off = alpha_n;
    end

% Use only valid Cl/Cd/phi entries
    if any(valid_idx)
        results(j,:) = [lam, ...
                        mean(Cl(valid_idx)), ...
                        mean(Cd(valid_idx)), ...
                        mean(phi_off(valid_idx)), ...
                        mean(Cl_new(valid_idx)), ...
                        mean(Cd(valid_idx))];
    end
end
T = array2table(results, ...
    'VariableNames', {'Lambda', 'Old_Cl', 'Old_Cd', 'Phi_deg', 'New_Cl', 'Corrected_New'});
disp(T);
%% Plot Results
figure;
plot(a_range, C_P_final, 'r', a_range, C_T_final, 'b-', 'LineWidth', 1.5);
xlabel('Axial Induction Factor a'); ylabel('Coefficient'); title('C_P and C_T vs a');
legend('C_P','C_T'); grid on;
figure;
plot(r_over_R, C_over_R_design, 'LineWidth', 1.5,'Marker', 'o');
xlabel('r/R'); ylabel('c/R'); title('Chord Distribution'); grid on;
figure;
plot(r_over_R, phi_design, 'LineWidth', 1.5,'Marker', 'x','Color', 'R');
xlabel('r/R'); ylabel('\phi (deg)'); title('Inflow Angle Distribution'); grid on;
figure;
plot(r_over_R, theta_design, 'LineWidth', 1.5 ,'Marker', '*','Color', 'm');
xlabel('r/R'); ylabel('\theta (deg)'); title('Pitch Angle Distribution'); grid on;
figure;
plot(r_over_R, theta_twist_design, 'LineWidth', 1.5,'Marker', '+','Color', 'G');
xlabel('r/R'); ylabel('\theta_{twist} (deg)'); title('Twist Distribution'); grid on;
%% Off-Design Angle Plot
figure; hold on;
for j = 1:length(Lamda_d)
    lamda_r_off = r_over_R * Lamda_d(j);
    alpha_off = alpha_design;
    error = ones(size(alpha_off));
```

```
  while max(error) > 0.01
    Cl = C_lo + C_l_alpha * alpha_off;
    Cd = C_do + K_1 * Cl + K_2 * Cl.^2;
    k = Cl ./ Cd;
    phi_off = theta_design + alpha_off;
    phi_rad = deg2rad(phi_off);

    a_dash = (((k .* tan(phi_rad)) - 1) .* (1 - lamda_r_off .* tan(phi_rad))) ./ ...
           ((lamda_r_off .* k) .* (1 + tan(phi_rad).^2));
    a = 1 - lamda_r_off .* tan(phi_rad) .* (1 + a_dash);

    exp_arg = -(B/2)*(1 - r_over_R)./(r_over_R.*sin(phi_rad));
    F = (2/pi) * acos(exp(exp_arg));

    tt2 = K_2 .* sin(phi_rad);
    tt1 = cos(phi_rad) + K_1 .* sin(phi_rad);
    tt0 = C_do .* sin(phi_rad) - ...
        (8*pi*F.*a.*r_over_R.*(sin(phi_rad)).^2) ./ ...
        (B*C_over_R_design.*(1-a));

    disc = tt1.^2 - 4 * tt0 .* tt2;
    valid_idx = disc >= 0 & abs(tt2) > eps;

    Cl_new = NaN(size(tt1));
    Cl_new(valid_idx) = (-tt1(valid_idx) + sqrt(disc(valid_idx))) ./ (2*tt2(valid_idx));

    alpha_c = (Cl_new - C_lo) / C_l_alpha;
    alpha_n = omega_r * alpha_c + (1 - omega_r) * alpha_off;
    error = abs(alpha_n - alpha_off);
    alpha_off = alpha_n;
  end

  plot(r_over_R, alpha_off, 'LineWidth', 1.2,'Marker', 'x', ...
    'DisplayName', ['\lambda = ', num2str(Lamda_d(j))]);
end
xlabel('r/R'); ylabel('\alpha'); title('r/R Vs. different values of \alpha');
legend('show', 'Location', 'best'); grid on;
figure;
plot(Lamda_d, Cp, 'r-o', Lamda_d, Ct, 'b-o', 'LineWidth', 1.5 ,'Marker', 'o');
xlabel('Tip Speed Ratio \lambda'); ylabel('Coefficient'); title('C_P and C_T vs \lambda');
legend('C_P','C_T'); grid on;
```